Plan for Direction C & D using LVLM:

Video Signature Vector v1

0) Pipeline overview (what you'll implement)

Input: local video path
→ upload to GCS
→ Google Video Intelligence metadata (shots/labels/objects)
→ compute Structured Signature (D)
→ extract keyframes (per shot / per interval)
→ LVLM query on keyframes (C)
→ parse LVLM text into Semantic Signature (C)
→ Fuse into one final signature object
→ save JSON: `VideoSignature_v1.json`

1) Structured Signature (Direction D)

This is the core mathematical vector derived from GCP metadata.

1.1 Feature vector (numeric)

Define:

```
S_struct = [f1, f2, f3, f4, f5, f6]
```

Where:

1. shot_count
2. scene_change_frequency = shot_count / duration_seconds
3. avg_shot_length (already computed)
4. shot_variance (already computed)
5. human_presence_ratio (already computed)
6. object_entropy (already computed)

These are strong because they're:

- stable across videos
- interpretable
- easy to justify academically

1.2 Optional "object profile"

To keep compression but preserve meaning:

- `top_objects` (already computed)
- `object_frequency_distribution` (already computed)

This isn't a vector element but is valuable for interpretability / reporting.

2) Semantic Signature (Direction C)

This comes from LVLM. It must be structured and normalized, not free text.

2.1 What you ask the LVLM

For each keyframe (or per shot), ask for:

- What is happening (short action)
- Emotional tone (choose from a fixed set)
- Who is interacting (yes/no + type)
- Environment (indoor/outdoor + location type)

2.2 Controlled vocabularies (important)

To make it scientific and consistent, you will force the model into predefined options.

Example sets:

emotion_class

- calm, tense, joyful, sad, fearful, angry, neutral, suspenseful

interaction_type

- none, conversation, conflict, cooperation, group_activity, unknown

environment

- indoor, outdoor, mixed, unknown

scene_type

- dialogue, action, landscape, social, object_focus, unknown

2.3 Semantic vector

Define:

```
S_sem = [e, i, env, st]
```

But since these are categorical, encode them as:

- One-hot vectors OR
- Integer IDs + mapping table

For v1 I recommend: store as strings + IDs, because it's easier and still formal.

Example:

```json
"semantic": {
  "emotion": { "label": "calm", "id": 0, "confidence": 0.72 },
  "interaction": { "label": "conversation", "id": 1, "confidence": 0.65 },
  "environment": { "label": "indoor", "id": 0, "confidence": 0.9 },
  "scene_type": { "label": "dialogue", "id": 0, "confidence": 0.66 }
}
```

3) Fusion Signature (D + C Combined)

This is your final "compressed semantic signature".

3.1 Final signature object (recommended format)

```json
{
  "version": "v1",
  "video_uri": "...",
  "duration_seconds": ...,

  "structured": {
    "shot_count": ...,
    "scene_change_frequency": ...,
    "avg_shot_length": ...,
    "shot_variance": ...,
    "human_presence_ratio": ...,
    "object_entropy": ...,
    "interaction_density_tracks_per_sec": ...,
    "top_objects": [...],
    "object_frequency_distribution": {...}
  },

  "semantic": {
    "emotion": { "label": "...", "id": ..., "confidence": ... },
    "interaction": { "label": "...", "id": ..., "confidence": ... },
    "environment": { "label": "...", "id": ..., "confidence": ... },
    "scene_type": { "label": "...", "id": ..., "confidence": ... },
    "summary": "1-2 short sentences (optional)"
  },

  "fusion": {
    "signature_vector": [
      scene_change_frequency,
      avg_shot_length,
      shot_variance,
      human_presence_ratio,
      object_entropy,
      interaction_density_tracks_per_sec,
      emotion_id,
      interaction_id,
      environment_id,
      scene_type_id
    ]
  }
}
```

This "fusion.signature_vector" is the actual compressed vector you can use for clustering, similarity, classification later.

4) Plug your current example into v1 (based on your JSON)

From your clip:

- duration = 8.583333
- shot_count = 4
- scene_change_frequency = 4 / 8.583333 = ~0.466 (cuts/sec)
- avg_shot_length = 2.114583
- shot_variance = 0.652235
- human_presence_ratio = 0.932039
- object_entropy = 3.236849
- interaction_density = 3.262136

This structured vector alone already describes:
Indoor, human-centric, moderate complexity, calm editing.

Semantic layer (later with LVLM) should likely produce:

- emotion: calm / neutral
- interaction: conversation
- environment: indoor
- scene_type: dialogue

That's exactly the C+D bridge.

5) Keyframe extraction rule (so C is systematic)

To keep it simple and formal:

Keyframes v1:

- pick 1 keyframe per shot at midpoint time

So for each shot:

```
keyframe_time = (start + end) / 2
```

This makes your LVLM analysis reproducible.

6) What makes your project "unique" (advisor wording)

"We propose a hybrid video abstraction method combining deterministic (חד ערכי) structured metadata from Google Video Intelligence with LVLM-generated semantic reasoning. The output is a compact signature vector representing both temporal(זמני)/visual statistics and high-level scene understanding."

Next step (no extra questions, just execution path)

If you want, I'll give you the exact code missions for Week 3:

1. Extend your script to extract keyframes automatically (OpenCV) using shot midpoints
2. Save keyframes to a folder
3. Create a `lvm_prompt.txt` template with controlled vocab answers
4. Stub a function `run_lvlm_on_keyframes()` that you can connect to LLaVA-Video later
5. Produce the full `VideoSignature_v1.json`